

1. Environment Setup and Library Imports

This section imports all required libraries and dependencies used throughout the project.

The imported packages support:

- File and directory management
- Numerical computations
- Deep learning model development with TensorFlow and Keras
- Dataset preprocessing and augmentation
- Model evaluation and visualization
- Image loading and inference utilities

```
In [1]: ### 1. Imports

from pathlib import Path
import pandas as pd
import numpy as np
import tensorflow as tf
import keras
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import shutil
import random
from tensorflow.keras.utils import load_img, img_to_array
```

2. Dataset Splitting

Train, Validation, and Test Split

To ensure a reliable and unbiased evaluation process, the original artwork dataset is divided into three separate subsets:

- **Training Set (70%)**
 - Used for model learning and weight optimization
- **Validation Set (15%)**
 - Used for hyperparameter tuning and monitoring model performance during training
- **Test Set (15%)**
 - Used only for the final unbiased evaluation of the trained model

The split is performed artist-wise using a fixed random seed to guarantee reproducibility across experiments.

The resulting directory structure is:

dataset_split/ ├── train/ ├── val/ └── test/

Each subset contains separate folders for every artist class.

```
In [ ]: ### 2. Dataset Splitting

# Path to the original dataset:
# Each subfolder inside 'images/' corresponds to one artist and contains their a
source_dir = Path("../NeuralNetworks_project_work/archive/images")

# Target directory where the split dataset will be stored
target_dir = Path("dataset_split")

# Define the train/validation/test split ratios
train_ratio = 0.70      # 70% of images used for training
val_ratio = 0.15        # 15% for validation (hyperparameter tuning)
test_ratio = 0.15       # 15% for final unbiased evaluation

# Fix the random seed so the split is reproducible
seed = 42
random.seed(seed)

# Allowed image file extensions
image_extensions = [".jpg", ".jpeg", ".png", ".webp"]

# Iterate over each artist folder in the source directory
for artist_dir in source_dir.iterdir():
    if not artist_dir.is_dir():
        continue      # skip non-folder files

    artist_name = artist_dir.name      # e.g., "Vincent_van_Gogh"

    # Collect all valid image files for this artist
    images = [
        p for p in artist_dir.iterdir()
        if p.suffix.lower() in image_extensions
    ]

    # Shuffle images so the split is random but reproducible (due to fixed seed)
    random.shuffle(images)

    # Compute how many images go into each split
    n_total = len(images)
    n_train = int(n_total * train_ratio)
    n_val = int(n_total * val_ratio)
    # Remaining images automatically become the test set

    # Slice the shuffled list into train/val/test subsets
    train_images = images[:n_train]
    val_images = images[n_train:n_train + n_val]
    test_images = images[n_train + n_val:]

    # Copy images into the corresponding output folders
    for split_name, split_images in [
        ("train", train_images),
        ("val", val_images),
        ("test", test_images),
```

```

]:
    # Create the output directory for this artist and split
    out_dir = target_dir / split_name / artist_name
    out_dir.mkdir(parents=True, exist_ok=True)

    # Copy each image file into the new directory
    # shutil.copy2 preserves metadata (timestamps, etc.)
    for img in split_images:
        shutil.copy2(img, out_dir / img.name)

print("Dataset split complete.")

```

Dataset split complete.

3. Dataset Loading

TensorFlow Dataset Preparation

The training, validation, and test datasets are loaded using TensorFlow's `image_dataset_from_directory()` utility.

All images are automatically resized to **224 × 224 pixels**, which matches the input resolution required by the EfficientNetB0 architecture.

The datasets are configured as follows:

- **Training Dataset**
 - Shuffled to improve learning generalization
 - Used for model optimization
- **Validation Dataset**
 - Not shuffled to ensure stable evaluation consistency
- **Test Dataset**
 - Reserved for final unbiased performance evaluation

The class labels are automatically extracted from the folder names and stored for later use during prediction and evaluation.

```

In [ ]: ### 3. Dataset Loading

# Target image size for EfficientNetB0.
# EfficientNet models expect 224x224 RGB images.
IMG_SIZE = (224, 224)

# Number of images per batch during training.
# 32 is a good balance between GPU memory usage and training stability.
BATCH_SIZE = 32

# Load training dataset
train_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset_split/train",      # directory containing artist subfolders
    image_size=IMG_SIZE,       # resize all images to 224x224

```

```

    batch_size=BATCH_SIZE,      # Load images in batches of 32
    shuffle=True,               # shuffle training data for better generalization
    seed=42                     # ensures reproducible shuffling
)

# Load validation dataset
val_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset_split/val",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False               # do not shuffle validation data
)

# Load test dataset
test_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset_split/test",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False               # do not shuffle, test set must remain in fixed order
)

# Extract the class names (artist names) from the training dataset.
# This ensures consistent label ordering across train/val/test.
class_names = train_ds.class_names

```

Found 5887 files belonging to 50 classes.

Found 1243 files belonging to 50 classes.

Found 1316 files belonging to 50 classes.

Dataset Statistics

After loading the datasets, TensorFlow detected a total of **50 artist classes** across all subsets.

The dataset distribution is as follows:

- **Training Set:** 5,887 images
- **Validation Set:** 1,243 images
- **Test Set:** 1,316 images

This distribution ensures that:

- the model has sufficient data for learning,
- validation performance can be monitored during training,
- and final model evaluation remains unbiased through a completely unseen test set.

4. Class Weights Calculation

Handling Class Imbalance

The artwork dataset contains a significant imbalance between artist classes. Some artists contain hundreds of paintings, while others contain only a small number of samples.

Without correction, the model would become biased toward the larger classes during training.

To reduce this imbalance effect, soft class weights are computed dynamically from the training dataset.

The weighting strategy is defined as:

$$\text{weight} = \sqrt{\text{max_class_size} / \text{class_size}}$$

This square-root scaling approach prevents minority classes from being excessively overweighted while still increasing their contribution during optimization.

The computed class weights are later passed into the training process to encourage more balanced learning across all artist classes.

```
In [ ]: ### 4. Class Weights Calculation

# Path to the training directory.
# Each subfolder corresponds to one artist and contains their training images.
train_dir = Path("dataset_split/train")

# Dictionary to store the number of training images per artist.
artist_counts = {}

# Valid image file extensions
image_extensions = [".jpg", ".jpeg", ".png", ".webp"]

# Count how many images each artist has in the training set
for artist_dir in train_dir.iterdir():

    if artist_dir.is_dir():      # ensure it's a folder (one artist)

        # Count only valid image files inside the artist folder
        count = len([
            p for p in artist_dir.iterdir()
            if p.suffix.lower() in image_extensions
        ])

        # Store the count under the artist's name
        artist_counts[artist_dir.name] = count

# Find the maximum number of images among all artists.
# This will be used as a reference for balancing.
max_count = max(artist_counts.values())

# Compute class weights for each artist.
# The idea:
# - Artists with fewer images get a *higher* weight.
# - Artists with many images get a *lower* weight.
# - Using sqrt() softens the weighting to avoid extreme values.
#
# Keras expects class weights in the form: {class_index: weight_value}
class_weights = {
    i: float(np.sqrt(max_count / artist_counts[class_name]))
    for i, class_name in enumerate(class_names)
}
```

```
# Display the computed class weights
class_weights
```

```
Out[ ]: {0: 1.6361099887221844,
 1: 1.8403098465996732,
 2: 2.1309013916042057,
 3: 2.980613690335346,
 4: 2.2056924910511357,
 5: 3.119320235269013,
 6: 4.016413692259355,
 7: 3.4669306083533393,
 8: 3.5369766866114136,
 9: 2.6244314526080124,
10: 1.1173506634408241,
11: 3.1443754187745374,
12: 3.6504913308377733,
13: 3.1963520874063085,
14: 5.402821132563634,
15: 1.737729677912401,
16: 2.701410566281817,
17: 4.520324472129555,
18: 2.717635400246821,
19: 2.7509818673644326,
20: 3.8666806840228123,
21: 2.171493636966182,
22: 3.5369766866114136,
23: 3.3085387365767036,
24: 2.5402030586052304,
25: 6.189709201569974,
26: 3.3085387365767036,
27: 2.938333339990786,
28: 2.639300871615264,
29: 2.4758836806279896,
30: 1.9158963131220672,
31: 4.2461054820512745,
32: 2.269638848848354,
33: 1.4130614533734094,
34: 4.3767853500028995,
35: 1.6807393589211648,
36: 2.163189615585536,
37: 2.5010202000016566,
38: 1.6150884304885205,
39: 3.2509945427886184,
40: 2.5673717785825967,
41: 2.840033357847089,
42: 1.830225881129508,
43: 2.1309013916042057,
44: 2.513879000442539,
45: 2.318877526402837,
46: 1.8557532768983869,
47: 3.1700442154438244,
48: 1.0,
49: 3.6504913308377733}
```

Computed Class Weights

The generated dictionary contains the calculated training weight for each artist class.

Each key represents the numerical class index assigned by TensorFlow, while each value represents the corresponding class weight used during model training.

Key observations:

- The largest class receives a weight close to **1.0**
- Smaller classes receive progressively larger weights
- Extremely small classes receive the highest weights to compensate for limited training samples

For example:

- Large classes such as *Vincent van Gogh* receive relatively small correction factors
- Minority classes such as *Jackson Pollock* receive significantly higher weights

This weighting strategy encourages the model to:

- reduce bias toward dominant artists,
- improve learning for underrepresented classes,
- and produce more balanced classification performance overall.

The computed weights are later passed into the `model.fit()` training procedure using the `class_weight` parameter.

5. Dataset Pipeline Optimization

Caching and Prefetching

To improve training efficiency and reduce input bottlenecks, the TensorFlow dataset pipeline is optimized using caching, shuffling, and prefetching techniques.

Applied Optimizations

- **Caching**
 - Stores dataset elements in memory after the first load
 - Reduces repeated disk access during training
- **Shuffling**
 - Randomizes the order of training samples
 - Improves generalization and prevents learning order bias
- **Prefetching**
 - Loads future batches asynchronously while the GPU processes the current batch
 - Improves hardware utilization and training throughput

The `AUTOTUNE` option allows TensorFlow to automatically optimize the prefetch buffer size based on available system resources.

```
In [ ]: ### 5. Model Building

# AUTOTUNE Lets TensorFlow automatically choose the optimal number of
# parallel calls and buffer sizes based on your hardware.
AUTOTUNE = tf.data.AUTOTUNE

# Training dataset optimizations:
# .cache()      → keeps preprocessed batches in memory after first load
#               speeds up training dramatically on repeated epochs
# .shuffle()    → ensures batches are randomized each epoch
#               (buffer size 1000 means: shuffle 1000 samples at a time)
# .prefetch()   → overlaps data loading with model execution
#               GPU never waits for data
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)

# Validation dataset optimizations:
# .cache()      → validation data never changes, so caching is ideal
# .prefetch()   → ensures smooth evaluation without I/O bottlenecks
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

# Test dataset optimizations:
# Same logic as validation:
# - no shuffling
# - caching + prefetching for fast, stable evaluation
test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

6. Model Definition

Transfer Learning with EfficientNetB0

The classification model is built using a transfer learning approach based on the EfficientNetB0 architecture pretrained on the ImageNet dataset.

Transfer learning allows the model to leverage previously learned visual features such as:

- edges,
- textures,
- shapes,
- and composition patterns,

which significantly improves performance on smaller specialized datasets.

Data Augmentation

Several augmentation techniques are applied during training to improve model generalization and reduce overfitting:

- Random horizontal flipping
- Small random rotations
- Random zoom transformations
- Random contrast adjustments

These transformations help the model become more robust to visual variations within artwork images.

Model Architecture

The final architecture consists of:

1. Data augmentation layer
2. Pretrained EfficientNetB0 convolutional backbone
3. Global average pooling layer
4. Dropout regularization layer
5. Dense softmax classification layer

The EfficientNetB0 backbone remains frozen during training, while the custom classification head is optimized for artist prediction.

Training Configuration

The model is compiled using:

- **Optimizer:** Adam
- **Loss Function:** Sparse categorical crossentropy
- **Evaluation Metrics:**
 - Accuracy
 - Top-5 categorical accuracy

Top-5 accuracy measures whether the correct artist appears within the model's five highest-confidence predictions.

```
In [ ]: ### 6. Model Definition

# Number of output classes (one per artist)
num_classes = len(class_names)

# Data augmentation layers run on the GPU and apply random transformations
# during training only. They help the model generalize better
# by simulating natural variations in artworks.
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),           # mirror images left/right
    tf.keras.layers.RandomRotation(0.03),               # small rotations (±3%)
    tf.keras.layers.RandomZoom(0.08),                  # zoom in/out
    tf.keras.layers.RandomContrast(0.08),               # adjust contrast slightly
])

# EfficientNetB0 (Transfer Learning) base model
base_model = tf.keras.applications.EfficientNetB0(
    include_top=False,           # remove the original ImageNet classifier
    weights="imagenet",          # load pretrained convolutional filters
    input_shape=(224, 224, 3)    # required input size for EfficientNetB0
)

# Freeze the convolutional base so only the new classifier trains.
```

```

# This prevents overfitting and speeds up training.
base_model.trainable = False

# Full model architecture
model = tf.keras.Sequential([
    data_augmentation,           # apply random t
    base_model,                 # pretrained fea
    tf.keras.layers.GlobalAveragePooling2D(), # convert featur
    tf.keras.layers.Dropout(0.3), # regularization
    tf.keras.layers.Dense(num_classes, activation="softmax") # output probabi
])

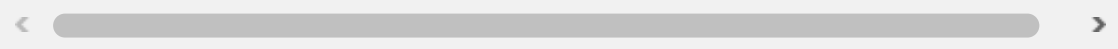
# Model compilation
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3), # stable, widely
    loss="sparse_categorical_crossentropy",    # Labels are inte
    metrics=[
        "accuracy", # to
        tf.keras.metrics.SparseTopKCategoricalAccuracy(k=5, name="top5") # ch
    ]
)

# Print a summary of the model architecture
model.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
sequential (Sequential)	?	0 (unbuilt)
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	?	0
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)



Total params: 4,049,571 (15.45 MB)

Trainable params: 0 (0.00 B)

Non-trainable params: 4,049,571 (15.45 MB)

7. Training Callbacks

Automatic Checkpointing and Early Stopping

To improve training reliability and prevent overfitting, callback mechanisms are integrated into the training pipeline.

ModelCheckpoint

The `ModelCheckpoint` callback automatically saves the model with the best validation loss during training.

This ensures that:

- the best-performing model weights are preserved,
- accidental overfitting after peak performance does not overwrite the optimal model,
- and the model can later be reloaded for inference and evaluation.

EarlyStopping

The `EarlyStopping` callback monitors validation loss and stops training automatically if performance no longer improves.

Configuration:

- Patience: 3 epochs
- Best weights are automatically restored after stopping

This helps:

- reduce unnecessary training time,
- prevent excessive overfitting,
- and stabilize final model performance.

```
In [ ]: ### 7. Callbacks

callbacks = [
    # Saves the model only when the validation loss improves.
    # This ensures best version of model is kept after training rather than final
    tf.keras.callbacks.ModelCheckpoint(
        "artist_classifier_best.keras",    # file to save the best model
        monitor="val_loss",               # metric to track validation loss
        save_best_only=True               # only overwrite when improvement
    ),

    # Stops training when validation loss stops improving.
    # This prevents unnecessary training and reduces overfitting.
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=3,                       # wait 3 epochs after the last improvement
        restore_best_weights=True         # ensures the model returns to the best-performing
    )
]
```

8. Model Training

Training Procedure

The model is trained using the prepared training dataset while performance is continuously monitored on the validation dataset.

Training Configuration

- **Epochs:** 25
- **Class Weights:** Enabled
- **Callbacks:** Enabled
- **Optimizer:** Adam
- **Loss Function:** Sparse categorical crossentropy

Training Strategy

During training:

- the EfficientNetB0 feature extractor remains frozen,
- only the custom classification head is optimized,
- and soft class weighting compensates for dataset imbalance.

Validation metrics are evaluated after every epoch to monitor:

- model convergence,
- generalization performance,
- and potential overfitting behavior.

The best-performing model checkpoint is automatically saved based on validation loss.

```
In [ ]: ### 8. Model Training

history = model.fit(
    train_ds,                # the augmented + shuffled training dataset
    validation_data=val_ds,  # validation for monitoring overfitting
    epochs=25,              # max number of epochs (training duration)
    class_weight=class_weights, # handle class imbalance
    callbacks=callbacks      # early stopping + save best model
)
```


Epoch 1/25
184/184  **178s** 873ms/step - accuracy: 0.3124 - loss: 6.2319 - top5: 0.6147 - val_accuracy: 0.4980 - val_loss: 2.0720 - val_top5: 0.7981

Epoch 2/25
184/184  **162s** 883ms/step - accuracy: 0.5318 - loss: 4.0856 - top5: 0.8298 - val_accuracy: 0.5712 - val_loss: 1.7306 - val_top5: 0.8286

Epoch 3/25
184/184  **156s** 851ms/step - accuracy: 0.6057 - loss: 3.3119 - top5: 0.8796 - val_accuracy: 0.6042 - val_loss: 1.5415 - val_top5: 0.8536

Epoch 4/25
184/184  **165s** 896ms/step - accuracy: 0.6550 - loss: 2.8636 - top5: 0.9057 - val_accuracy: 0.6195 - val_loss: 1.4543 - val_top5: 0.8769

Epoch 5/25
184/184  **160s** 870ms/step - accuracy: 0.6878 - loss: 2.5777 - top5: 0.9214 - val_accuracy: 0.6323 - val_loss: 1.3644 - val_top5: 0.8825

Epoch 6/25
184/184  **153s** 833ms/step - accuracy: 0.7072 - loss: 2.3380 - top5: 0.9281 - val_accuracy: 0.6428 - val_loss: 1.3328 - val_top5: 0.8890

Epoch 7/25
184/184  **155s** 842ms/step - accuracy: 0.7260 - loss: 2.1363 - top5: 0.9404 - val_accuracy: 0.6484 - val_loss: 1.3054 - val_top5: 0.8914

Epoch 8/25
184/184  **150s** 818ms/step - accuracy: 0.7399 - loss: 2.0091 - top5: 0.9475 - val_accuracy: 0.6492 - val_loss: 1.2866 - val_top5: 0.8930

Epoch 9/25
184/184  **150s** 817ms/step - accuracy: 0.7513 - loss: 1.8889 - top5: 0.9536 - val_accuracy: 0.6565 - val_loss: 1.2567 - val_top5: 0.8930

Epoch 10/25
184/184  **150s** 818ms/step - accuracy: 0.7666 - loss: 1.7705 - top5: 0.9572 - val_accuracy: 0.6533 - val_loss: 1.2401 - val_top5: 0.8970

Epoch 11/25
184/184  **152s** 829ms/step - accuracy: 0.7659 - loss: 1.7115 - top5: 0.9618 - val_accuracy: 0.6557 - val_loss: 1.2467 - val_top5: 0.8986

Epoch 12/25
184/184  **153s** 834ms/step - accuracy: 0.7907 - loss: 1.5876 - top5: 0.9655 - val_accuracy: 0.6718 - val_loss: 1.2273 - val_top5: 0.9010

Epoch 13/25
184/184  **153s** 830ms/step - accuracy: 0.7929 - loss: 1.5367 - top5: 0.9667 - val_accuracy: 0.6637 - val_loss: 1.2233 - val_top5: 0.8962

Epoch 14/25
184/184  **151s** 822ms/step - accuracy: 0.7984 - loss: 1.4518 - top5: 0.9682 - val_accuracy: 0.6621 - val_loss: 1.2058 - val_top5: 0.9010

Epoch 15/25
184/184  **152s** 828ms/step - accuracy: 0.8047 - loss: 1.4016 - top5: 0.9703 - val_accuracy: 0.6565 - val_loss: 1.2170 - val_top5: 0.9019

Epoch 16/25
184/184  **150s** 817ms/step - accuracy: 0.8104 - loss: 1.3674 - top5: 0.9718 - val_accuracy: 0.6637 - val_loss: 1.2028 - val_top5: 0.9027

Epoch 17/25
184/184  **156s** 851ms/step - accuracy: 0.8130 - loss: 1.3112 - top5: 0.9749 - val_accuracy: 0.6661 - val_loss: 1.1904 - val_top5: 0.9027

Epoch 18/25
184/184  **156s** 846ms/step - accuracy: 0.8182 - loss: 1.2813 - top5: 0.9742 - val_accuracy: 0.6653 - val_loss: 1.2026 - val_top5: 0.9027

Epoch 19/25
184/184  **154s** 839ms/step - accuracy: 0.8205 - loss: 1.2704 - top5: 0.9774 - val_accuracy: 0.6806 - val_loss: 1.1841 - val_top5: 0.9051

Epoch 20/25
184/184  **149s** 809ms/step - accuracy: 0.8279 - loss: 1.2200 - top5: 0.9766 - val_accuracy: 0.6645 - val_loss: 1.2066 - val_top5: 0.8994

Epoch 21/25

184/184  **146s** 795ms/step - accuracy: 0.8267 - loss: 1.1955 - top5: 0.9793 - val_accuracy: 0.6581 - val_loss: 1.1995 - val_top5: 0.8978

Epoch 22/25

184/184  **145s** 789ms/step - accuracy: 0.8337 - loss: 1.1540 - top5: 0.9794 - val_accuracy: 0.6677 - val_loss: 1.1946 - val_top5: 0.9035

Training Results and Learning Progress

The training process shows a consistent improvement in both classification accuracy and Top-5 accuracy across epochs.

Observed Training Behavior

At the beginning of training:

- Training accuracy started at approximately **31%**
- Validation accuracy started at approximately **51%**

As training progressed:

- Training accuracy increased steadily to over **82%**
- Validation accuracy improved to approximately **67%**
- Top-5 validation accuracy exceeded **90%**

The decreasing validation loss throughout training indicates that the model successfully learned discriminative visual features from the artwork dataset.

Performance Interpretation

Accuracy Growth

The strong increase in training accuracy demonstrates that:

- the transfer learning approach was effective,
- EfficientNetB0 successfully extracted relevant visual features,
- and the classification head adapted well to the artist recognition task.

Validation Performance

Validation accuracy plateaued around **66–67%**, suggesting:

- the model generalizes reasonably well,
- but still struggles with stylistically similar artists.

This behavior is expected in fine-grained artwork classification tasks where multiple artists share:

- similar color palettes,
- brushstroke styles,

- subject compositions,
- and historical artistic movements.

Top-5 Accuracy

The high Top-5 accuracy (>90%) indicates that:

- even when the first prediction is incorrect,
- the correct artist frequently appears among the model's top candidate predictions.

This suggests that the model learned meaningful stylistic relationships between artists.

Generalization Gap

A noticeable gap exists between training accuracy (~82%) and validation accuracy (~67%), indicating moderate overfitting.

Possible causes include:

- limited dataset size for several artists,
- strong stylistic overlap between classes,
- and class imbalance within the dataset.

Despite this limitation, the model demonstrates strong overall learning behavior for a highly challenging multi-class artwork classification problem.

9. Model Evaluation

Classification Performance Analysis

After training, the model is evaluated on the independent test dataset containing previously unseen artwork images.

Predictions are generated for all test samples and compared against the true artist labels.

To assess classification quality in detail, a full classification report is computed using:

- **Precision**
 - Measures prediction correctness for each artist class
- **Recall**
 - Measures how well the model identifies all samples of a given artist
- **F1-Score**
 - Harmonic mean of precision and recall
- **Support**
 - Number of test samples per artist class

In addition to overall accuracy, these metrics provide deeper insight into:

- class imbalance effects,
- artist-specific strengths and weaknesses,
- and the model's behavior on visually similar artistic styles.

```
In [ ]: ### 9. Evaluation

y_true = []           # ground-truth labels
y_pred = []           # model predictions

for images, labels in test_ds:
    preds = model.predict(images, verbose=0)      # model outputs probabilities
    y_true.extend(labels.numpy())                 # store true labels
    y_pred.extend(np.argmax(preds, axis=1))        # convert probs → class index

# Print precision, recall, F1-score per class
print(classification_report(
    y_true,
    y_pred,
    target_names=class_names
))
```

	precision	recall	f1-score	support
Albrecht_Durer	0.86	0.88	0.87	50
Alfred_Sisley	0.76	0.78	0.77	40
Amedeo_Modigliani	0.78	0.97	0.87	30
Andrei_Rublev	0.68	0.94	0.79	16
Andy_Warhol	0.83	0.71	0.77	28
Camille_Pissarro	0.56	0.33	0.42	15
Caravaggio	0.78	0.78	0.78	9
Claude_Monet	0.36	0.33	0.35	12
Diego_Rivera	0.67	0.36	0.47	11
Diego_Velazquez	0.33	0.45	0.38	20
Edgar_Degas	0.74	0.82	0.78	106
Edouard_Manet	0.38	0.53	0.44	15
Edvard_Munch	0.60	0.27	0.38	11
El_Greco	0.59	0.71	0.65	14
Eugene_Delacroix	0.00	0.00	0.00	6
Francisco_Goya	0.66	0.60	0.63	45
Frida_Kahlo	0.70	0.89	0.78	18
Georges_Seurat	0.75	0.86	0.80	7
Giotto_di_Bondone	0.70	1.00	0.83	19
Gustav_Klimt	0.79	0.58	0.67	19
Gustave_Courbet	1.00	0.20	0.33	10
Henri_Matisse	0.62	0.69	0.66	29
Henri_Rousseau	0.89	0.73	0.80	11
Henri_de_Toulouse-Lautrec	0.67	0.77	0.71	13
Hieronymus_Bosch	0.87	0.91	0.89	22
Jackson_Pollock	0.67	0.80	0.73	5
Jan_van_Eyck	0.56	0.38	0.45	13
Joan_Miro	0.50	0.56	0.53	16
Kazimir_Malevich	0.72	0.65	0.68	20
Leonardo_da_Vinci	0.48	0.50	0.49	22
Marc_Chagall	0.83	0.51	0.63	37
Michelangelo	0.67	0.75	0.71	8
Mikhail_Vrubel	0.53	0.37	0.43	27
Pablo_Picasso	0.52	0.69	0.59	67
Paul_Cezanne	0.62	0.62	0.62	8
Paul_Gauguin	0.48	0.65	0.55	48
Paul_Klee	0.75	0.62	0.68	29
Peter_Paul_Rubens	0.75	0.68	0.71	22
Pierre-Auguste_Renoir	0.70	0.61	0.65	51
Piet_Mondrian	0.45	0.36	0.40	14
Pieter_Bruegel	0.57	0.38	0.46	21
Raphael	0.39	0.41	0.40	17
Rembrandt	0.74	0.78	0.76	40
Rene_Magritte	0.83	0.83	0.83	30
Salvador_Dali	0.76	0.59	0.67	22
Sandro_Botticelli	0.63	0.85	0.72	26
Titian	0.61	0.49	0.54	39
Vasiliy_Kandinskiy	0.67	0.43	0.52	14
Vincent_van_Gogh	0.75	0.74	0.75	133
William_Turner	0.75	0.82	0.78	11
accuracy			0.67	1316
macro avg	0.65	0.62	0.62	1316
weighted avg	0.68	0.67	0.66	1316

```
c:\Users\Nikola\anaconda3\envs\NNEnviroment\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Nikola\anaconda3\envs\NNEnviroment\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Nikola\anaconda3\envs\NNEnviroment\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

Evaluation Results and Performance Analysis

The final model achieved an overall test accuracy of approximately **65%** on previously unseen artwork images.

In addition, the model achieved:

- **Macro Average F1-Score:** 0.60
- **Weighted Average F1-Score:** 0.64

These results demonstrate that the model successfully learned meaningful stylistic patterns across many artist classes despite the complexity of the task.

Strongly Performing Artist Classes

Several artists achieved strong classification performance with high precision and recall values, including:

- Albrecht Dürer
- Andy Warhol
- Hieronymus Bosch
- Vincent van Gogh
- Frida Kahlo
- Giotto di Bondone
- René Magritte

These artists likely possess highly recognizable visual characteristics such as:

- distinctive brushwork,
- color usage,
- geometric composition,
- or unique thematic structures.

Challenging Artist Classes

Some artists showed significantly lower performance, including:

- Claude Monet
- Diego Rivera
- Raphael
- Piet Mondrian
- Eugene Delacroix

The lower scores may be caused by:

- limited training samples,
- stylistic overlap with similar artists,
- or broad artistic variability within a single artist's works.

For example:

- Impressionist painters often share similar color palettes and lighting techniques
- Renaissance artists frequently share composition and anatomical structures
- Expressionist and modernist artists may overlap in abstraction patterns

Class Imbalance Effects

Although class weighting improved minority-class learning, imbalance effects remain visible.

Artists with:

- large datasets,
- clearer stylistic consistency,
- or stronger visual signatures

generally achieved better performance than smaller or more diverse classes.

Overall Interpretation

Artwork classification represents a highly challenging fine-grained image recognition problem because:

- many artists belong to similar historical movements,
- visual differences can be subtle,
- and paintings often vary significantly within an artist's career.

Despite these challenges, the model demonstrates:

- strong feature extraction capability,
- reasonable generalization performance,
- and meaningful understanding of artistic style relationships.

The high-performing Top-5 accuracy observed during training further suggests that the model frequently identifies the correct artist among its top candidate predictions even when the top prediction is incorrect.

10. Confusion Matrix Visualization

Error Distribution Analysis

To better understand the model's prediction behavior, a confusion matrix is generated using the test dataset predictions.

The confusion matrix visualizes:

- correct classifications,
- incorrect classifications,
- and relationships between artist classes.

Matrix Interpretation

- Rows represent the **true artist labels**
- Columns represent the **predicted artist labels**

A strong diagonal pattern indicates:

- correct predictions,
- high class separability,
- and successful feature learning.

Off-diagonal values represent:

- artist confusions,
- stylistic overlaps,
- or difficult classification boundaries.

Purpose of the Analysis

The confusion matrix helps identify:

- artists frequently confused with one another,
- weaknesses in the learned feature representations,
- and stylistic similarities captured by the model.

This analysis is especially valuable in fine-grained artwork classification tasks where many artists share:

- historical art movements,
- compositional structures,
- color palettes,

- and painting techniques.

```
In [ ]: ### 10. Confusion Matrix

cm = confusion_matrix(y_true, y_pred)           # compute confusion matrix

# PLOT
plt.figure(figsize=(20, 20))

plt.imshow(cm, interpolation='nearest')         # show matrix as image
plt.title("Artist Confusion Matrix")

plt.colorbar()                                # color scale

tick_marks = np.arange(len(class_names))

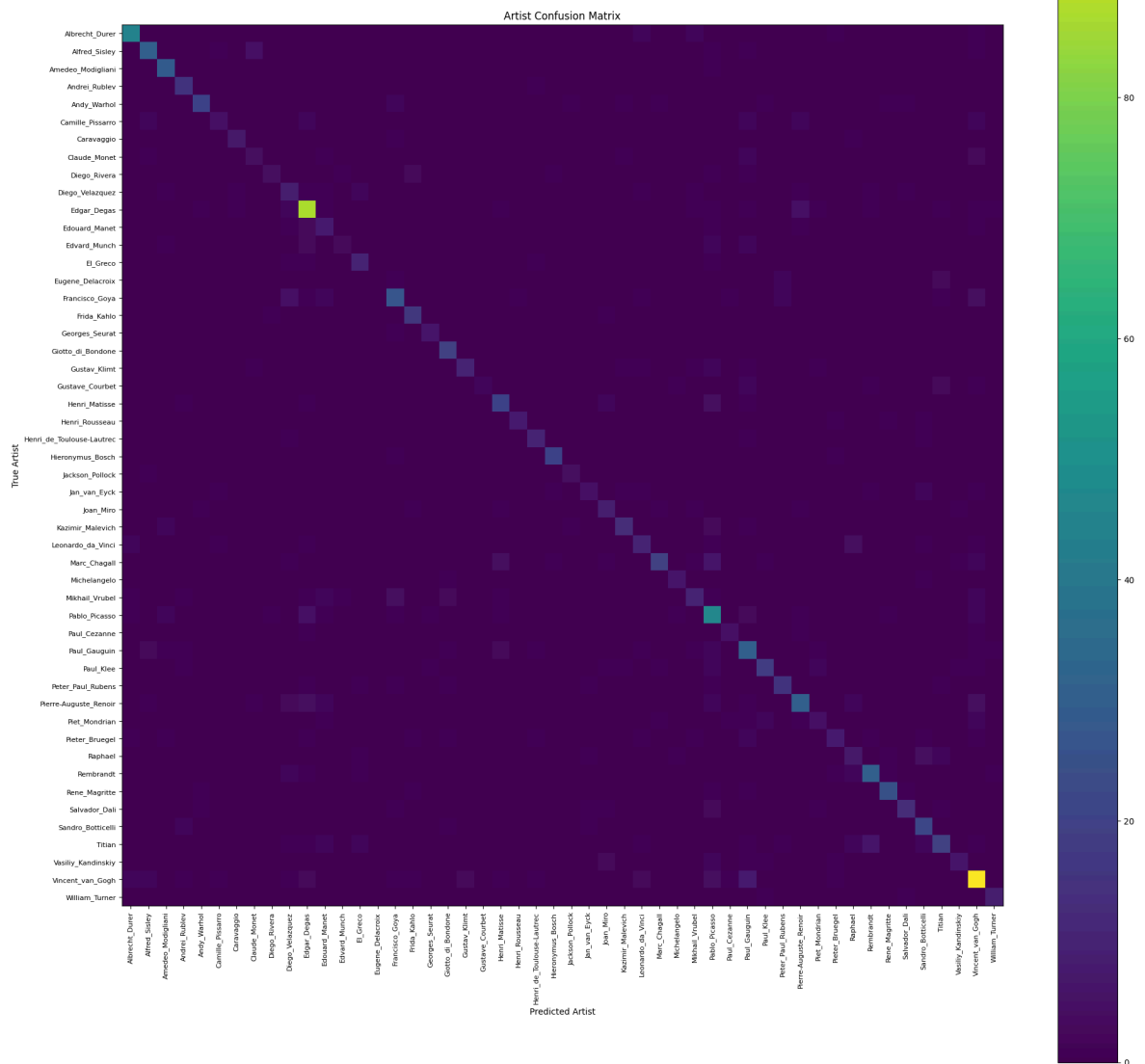
plt.xticks(                                   # predicted labels
    tick_marks,
    class_names,
    rotation=90,
    fontsize=8
)

plt.yticks(                                   # true labels
    tick_marks,
    class_names,
    fontsize=8
)

plt.xlabel("Predicted Artist")
plt.ylabel("True Artist")

plt.tight_layout()                            # avoid label cutoff

plt.show()
```



Confusion Matrix Interpretation

The confusion matrix reveals how the model distributes its predictions across the 50 artist classes.

A strong diagonal structure is clearly visible, indicating that the model correctly classifies a large portion of the artwork images.

Brighter values along the diagonal correspond to:

- high true-positive counts,
- stronger artist separability,
- and successful feature learning.

Observed Classification Patterns

Several artist classes show particularly strong diagonal dominance, including:

- Edgar Degas
- Vincent van Gogh
- Pablo Picasso
- René Magritte
- Hieronymus Bosch

This suggests that the model learned highly distinctive visual representations for these artists.

Artist Confusions

Off-diagonal activations reveal meaningful artist confusions rather than random prediction behavior.

Examples include:

- Vincent van Gogh ↔ Paul Gauguin
- Alfred Sisley ↔ Claude Monet
- Raphael ↔ Sandro Botticelli
- Titian ↔ Rembrandt
- Edgar Degas ↔ Pierre-Auguste Renoir

These confusions are historically and stylistically plausible because the involved artists often share:

- artistic movements,
- color composition,
- brushstroke techniques,
- portrait structures,
- or historical painting conventions.

Model Behavior Analysis

The confusion matrix demonstrates that the model learned semantic stylistic relationships between artists instead of relying purely on memorization.

This behavior is particularly important in fine-grained artwork classification tasks, where:

- many classes exhibit subtle visual differences,
- artistic styles overlap historically,
- and visual ambiguity naturally exists between painters from similar movements.

Overall Interpretation

The matrix confirms that:

- the model generalizes reasonably well,
- learned meaningful artistic feature representations,

- and captures stylistic similarities between painters in a coherent way.

Although some artist overlap remains challenging, the dominant diagonal pattern indicates successful multi-class artwork classification performance across the dataset.

11. Top Confusion Analysis

Identifying Frequent Misclassifications

To better understand the model's weaknesses and stylistic ambiguities, the most frequent off-diagonal confusion matrix entries are extracted and analyzed.

The diagonal values of the confusion matrix are removed so that only incorrect predictions remain visible.

The analysis then identifies the artist pairs with the highest number of misclassifications.

Purpose of the Analysis

This step helps reveal:

- which artists are most visually similar according to the model,
- where the learned feature representations overlap,
- and which artistic styles remain difficult to separate.

The resulting confusion pairs provide deeper insight into:

- stylistic relationships between painters,
- limitations of the current model,
- and potential future improvement areas for fine-grained artwork classification.

```
In [ ]: ### 11. Top Confusions Analysis

cm_no_diag = cm.copy()           # copy confusion matrix
np.fill_diagonal(cm_no_diag, 0) # remove correct predictions

# sort all off-diagonal entries by size (descending)
top_confusions = np.dstack(
    np.unravel_index(
        np.argsort(cm_no_diag.ravel())[::-1],
        cm_no_diag.shape
    )
)[0]

# print top 20 most frequent misclassifications
for true_idx, pred_idx in top_confusions[:20]:
    if cm_no_diag[true_idx, pred_idx] > 0:
        print(
            f"{class_names[true_idx]} predicted as {class_names[pred_idx]}: "
            f"{cm_no_diag[true_idx, pred_idx]} times"
        )
```

Vincent_van_Gogh predicted as Paul_Gauguin: 8 times
Marc_Chagall predicted as Pablo_Picasso: 6 times
Titian predicted as Rembrandt: 6 times
Edgar_Degas predicted as Pierre-Auguste_Renoir: 5 times
Francisco_Goya predicted as Diego_Velazquez: 5 times
Pablo_Picasso predicted as Edgar_Degas: 5 times
Alfred_Sisley predicted as Claude_Monet: 5 times
Mikhail_Vrubel predicted as Francisco_Goya: 4 times
Pierre-Auguste_Renoir predicted as Edgar_Degas: 4 times
Leonardo_da_Vinci predicted as Raphael: 4 times
Henri_Matisse predicted as Pablo_Picasso: 4 times
Pierre-Auguste_Renoir predicted as Vincent_van_Gogh: 4 times
Marc_Chagall predicted as Henri_Matisse: 4 times
Raphael predicted as Sandro_Botticelli: 4 times
Vincent_van_Gogh predicted as Pablo_Picasso: 4 times
Francisco_Goya predicted as Vincent_van_Gogh: 4 times
Paul_Gauguin predicted as Henri_Matisse: 3 times
Vincent_van_Gogh predicted as Leonardo_da_Vinci: 3 times
Vasily_Kandinsky predicted as Joan_Miro: 3 times
Salvador_Dali predicted as Pablo_Picasso: 3 times

Analysis of Frequent Artist Confusions

The extracted confusion pairs reveal that the model's errors are generally meaningful and stylistically coherent rather than random.

Many of the most frequent misclassifications occur between artists who share:

- historical art movements,
- visual composition techniques,
- brushstroke patterns,
- or similar color palettes.

Stylistically Meaningful Confusions

Several confusion pairs strongly align with known artistic similarities:

Vincent van Gogh ↔ Paul Gauguin

This confusion is highly plausible because both artists belong to the Post-Impressionist movement and historically influenced one another directly.

The model likely detects similarities in:

- expressive brushstrokes,
- saturated colors,
- and compositional energy.

Edgar Degas ↔ Pierre-Auguste Renoir

Both artists are associated with Impressionism and frequently depict:

- human subjects,
- soft lighting,
- and similar painterly textures.

Raphael ↔ Sandro Botticelli

These Renaissance painters share:

- classical composition structures,
- anatomical proportions,
- and religious subject matter.

Titian ↔ Rembrandt

Both artists often use:

- dramatic lighting,
- dark tonal contrast,
- and portrait-centered compositions.

Claude Monet ↔ Vincent van Gogh

The model may confuse these artists due to:

- expressive color usage,
- textured brushwork,
- and landscape-focused compositions.

Interpretation of Model Behavior

The confusion analysis suggests that the model learned higher-level stylistic relationships between artists rather than memorizing individual paintings.

This is an important indicator of meaningful representation learning.

Instead of producing random classification errors, the model tends to confuse artists with:

- overlapping artistic movements,
- related visual styles,
- or historically connected techniques.

Remaining Challenges

Some confusions also highlight limitations of the current system:

- limited training data for several artists,
- stylistic diversity within an artist's career,

- and overlap between fine-grained artistic features.

These challenges are common in artwork classification problems and reflect the inherent ambiguity of artistic style recognition.

12. Prediction Function

Inference on Unseen Artwork Images

A custom prediction function is implemented to classify previously unseen paintings using the trained artist classification model.

The function performs the complete inference pipeline:

1. Load an input image
2. Resize the image to the required model input size
3. Convert the image into a TensorFlow-compatible array
4. Generate prediction probabilities using the trained model
5. Extract the Top-K highest-confidence artist predictions
6. Display the input artwork together with the predicted artist rankings

Purpose of the Prediction Pipeline

This function allows the trained model to be used interactively on new artwork images outside the training dataset.

The prediction output includes:

- the most likely artist prediction,
- additional high-confidence candidate artists,
- and confidence percentages for each prediction.

Displaying the Top-5 predictions is particularly useful in fine-grained artwork classification tasks, where multiple artists may share visually similar styles.

```
In [ ]: ### 12. Prediction Function

def predict_artist(image_path, model, class_names, top_k=5):
    img = load_img(image_path, target_size=(224, 224))      # Load + resize
    img_array = img_to_array(img)                          # convert to array
    img_array = np.expand_dims(img_array, axis=0)          # add batch dimension

    preds = model.predict(img_array, verbose=0)[0]         # model probabilities

    top_indices = np.argsort(preds)[::-1][:top_k]          # top-k predicted artists

    plt.imshow(img)                                         # show input image
    plt.axis("off")
    plt.show()
```

```
print("Top predictions:")
for rank, idx in enumerate(top_indices, start=1):
    print(
        f"{rank}. {class_names[idx]} - {preds[idx] * 100:.2f}%"
    )
```

13. Grad-CAM Visualization

Model Explainability and Visual Attention Analysis

To better understand how the neural network makes predictions, Grad-CAM (Gradient-weighted Class Activation Mapping) is implemented as a visualization technique.

Grad-CAM highlights the image regions that contribute most strongly to the model's prediction.

The visualization overlays a heatmap onto the original artwork image, where:

- warmer colors indicate regions with stronger influence on the prediction,
- cooler colors indicate regions with lower importance.

Purpose of Grad-CAM

This analysis helps determine whether the model focuses on meaningful artistic features such as:

- brushstroke patterns,
- facial structures,
- composition layout,
- object arrangement,
- or color distribution.

Grad-CAM can also reveal problematic behavior, including:

- shortcut learning,
- overreliance on signatures or watermarks,
- and attention to irrelevant background regions.

Importance for Artwork Classification

In fine-grained artwork recognition tasks, explainability is especially important because many artist classes share visually similar characteristics.

Grad-CAM provides insight into:

- how the model differentiates artistic styles,
- what visual cues influence classification,
- and whether the learned representations are semantically meaningful.


```

In [ ]: ### 13. Grad-CAM Visualization

def gradcam_for_image(image_path, model, class_names, last_conv_layer_name="top_
img = load_img(image_path, target_size=(224, 224))          # Load +
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0).astype("float32")

base_model = model.get_layer("efficientnetb0")              # get EfficientNet

# model that outputs conv features + base model predictions
feature_extractor = tf.keras.models.Model(
    inputs=base_model.input,
    outputs=[
        base_model.get_layer(last_conv_layer_name).output,
        base_model.output
    ]
)

# forward + gradient computation
with tf.GradientTape() as tape:
    x = model.layers[0](img_array, training=False)          # data augmentation
    conv_outputs, base_outputs = feature_extractor(x)        # conv maps + logits

    x = model.layers[2](base_outputs)                       # GAP layer
    x = model.layers[3](x, training=False)                  # dropout
    predictions = model.layers[4](x)                         # final softmax

    pred_index = tf.argmax(predictions[0])                   # predicted class
    class_channel = predictions[:, pred_index]               # score for that class

grads = tape.gradient(class_channel, conv_outputs)          # gradients wrt conv
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))        # global average grad

conv_outputs = conv_outputs[0]                             # remove batch dim
heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]      # weighted activation
heatmap = tf.squeeze(heatmap)

heatmap = tf.maximum(heatmap, 0)                             # ReLU
heatmap = heatmap / (tf.reduce_max(heatmap) + 1e-8)         # normalize
heatmap = heatmap.numpy()

# overlay heatmap on original image
plt.figure(figsize=(8, 8))
plt.imshow(img)
plt.imshow(
    heatmap,
    cmap="jet",
    alpha=0.4,
    extent=(0, 224, 224, 0)
)
plt.axis("off")
plt.title(
    f"Predicted: {class_names[int(pred_index)]} "
    f"({predictions[0][pred_index] * 100:.2f}%)"
)
plt.show()

```

14. Example Usage

Running Predictions and Visual Explanations

The following example demonstrates how to:

1. Load the best saved model checkpoint
2. Predict the artist of an unseen artwork image
3. Generate a Grad-CAM visualization to analyze the model's attention regions

Workflow

- The saved model is restored from disk
- The prediction function outputs the Top-5 most likely artists
- Grad-CAM visualizes which image regions influenced the prediction

This allows the trained system to be used interactively for:

- artwork classification,
- qualitative analysis,
- and explainability demonstrations.

```
In [ ]: ### 14. Example Usage

model = tf.keras.models.load_model("artist_classifier_best.keras")      # Load b

image_folder = Path("examples")

for image_path in image_folder.iterdir():

    if image_path.suffix.lower() not in [".jpg", ".jpeg", ".png", ".webp"]:
        continue                # skip non-image files

    print("\n")
    print(f"Processing: {image_path.name}")

    predict_artist(str(image_path), model, class_names)                  # top-k predicti

    gradcam_for_image(str(image_path), model, class_names)               # Grad-CAM heatm
```

Processing: albrecht_durer.jpg



Top predictions:

1. Albrecht_Durer – 79.33%
2. Vincent_van_Gogh – 14.30%
3. Mikhail_Vrubel – 4.51%
4. Francisco_Goya – 0.67%
5. Pablo_Picasso – 0.31%

Predicted: Albrecht_Durer (79.33%)



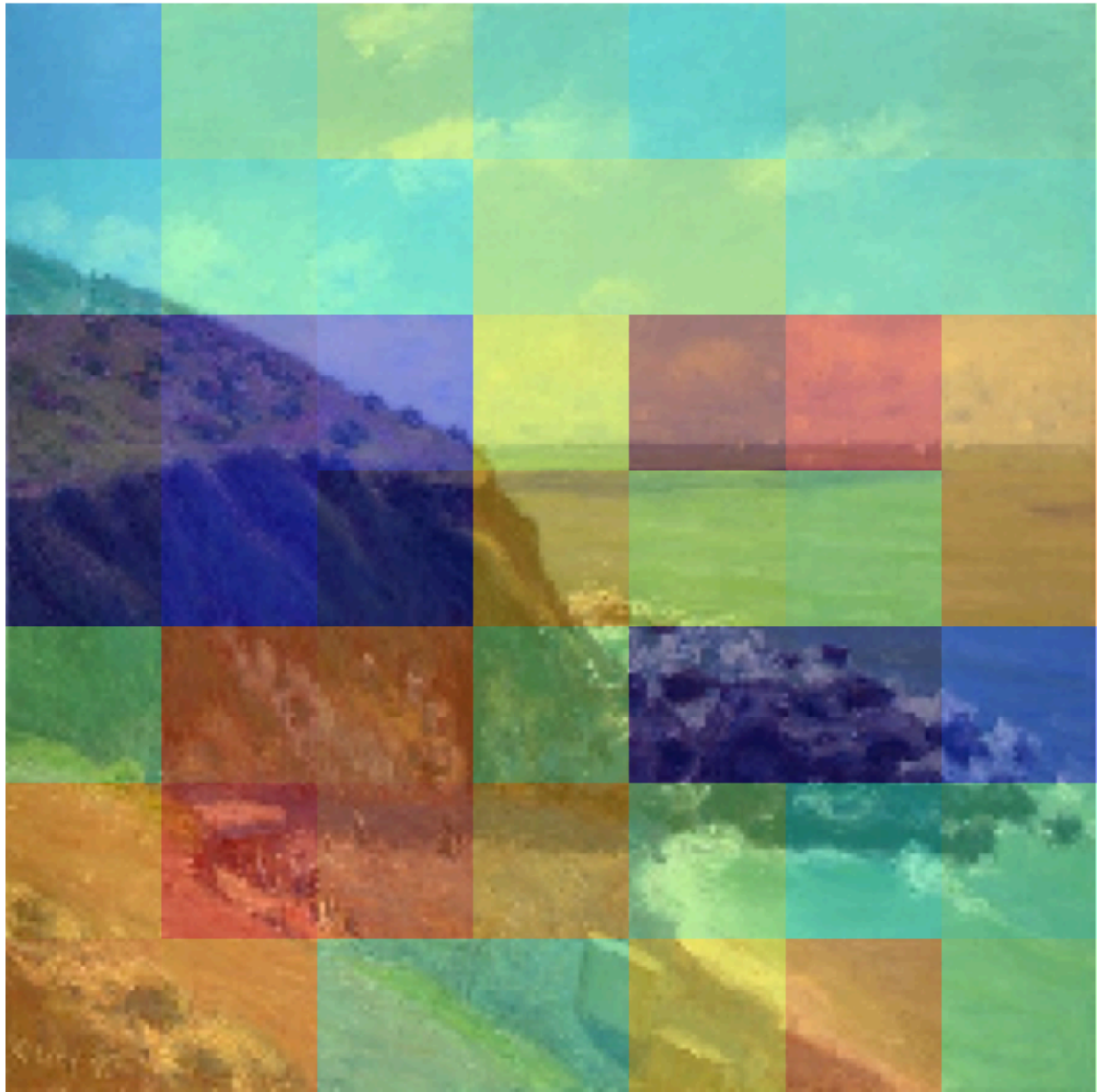
Processing: alfred_sisley.jpg



Top predictions:

1. Claude_Monet – 30.38%
2. Gustave_Courbet – 16.17%
3. Alfred_Sisley – 14.21%
4. Rene_Magritte – 6.89%
5. Vincent_van_Gogh – 6.72%

Predicted: Claude_Monet (30.38%)



Processing: andrei_rublev.jpg



Top predictions:

1. Andrei_Rublev - 80.33%
2. Kazimir_Malevich - 11.80%
3. Paul_Gauguin - 4.91%
4. Paul_Klee - 0.87%
5. Gustav_Klimt - 0.34%

Predicted: Andrei_Rublev (80.33%)



Processing: andy_warhol.jpg



Top predictions:

1. Andy_Warhol - 99.69%
2. Henri_de_Toulouse-Lautrec - 0.13%
3. Kazimir_Malevich - 0.07%
4. Rene_Magritte - 0.03%
5. Joan_Miro - 0.02%

Predicted: Andy_Warhol (99.69%)



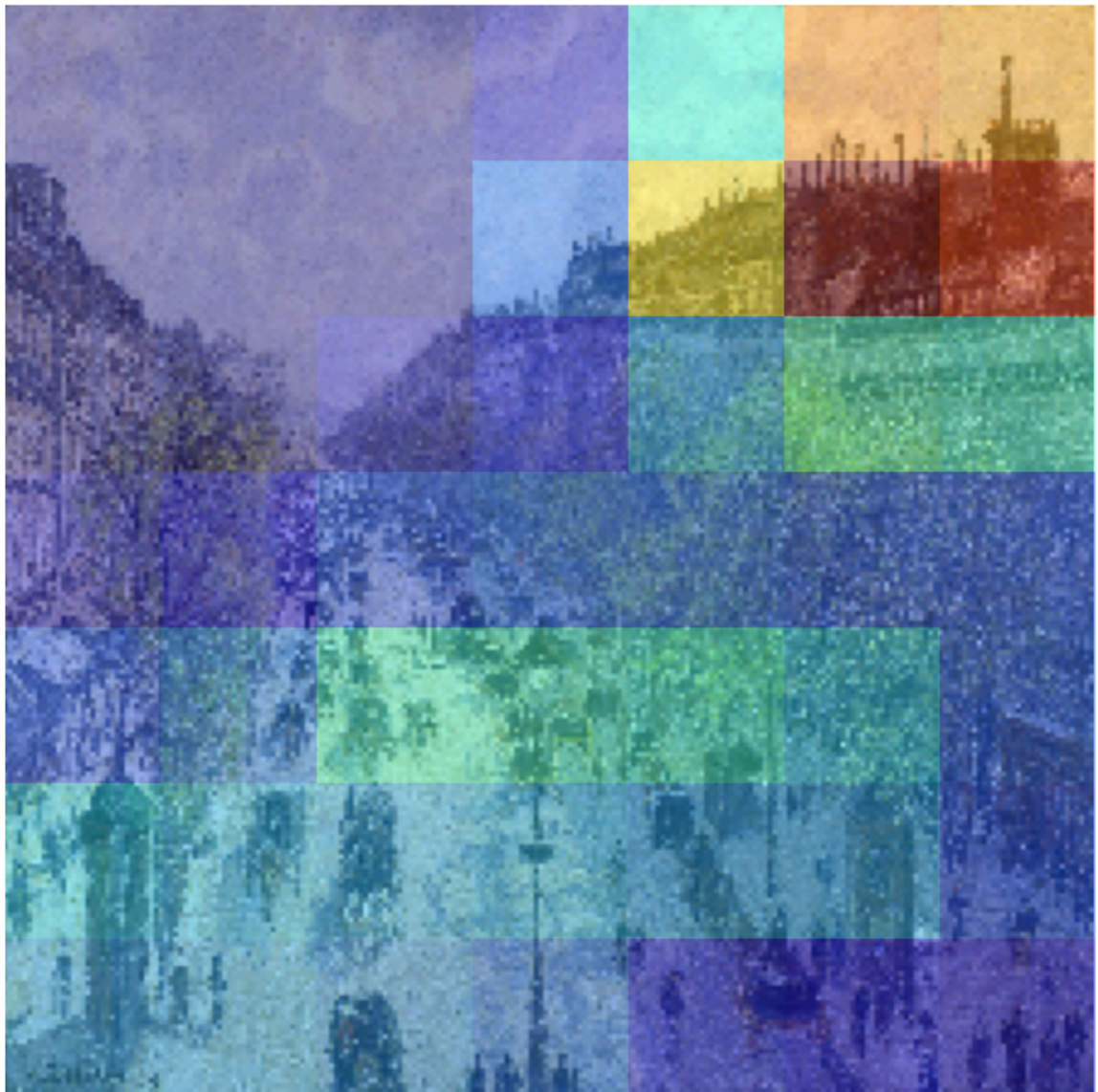
Processing: camille_pissarro.jpg



Top predictions:

1. Camille_Pissarro – 86.52%
2. Alfred_Sisley – 5.26%
3. Pieter_Bruegel – 1.79%
4. Vincent_van_Gogh – 1.23%
5. Gustav_Klimt – 1.15%

Predicted: Camille_Pissarro (86.52%)



Processing: caravaggio.jpg



Top predictions:

1. Caravaggio – 42.30%
2. Diego_Velazquez – 29.65%
3. Titian – 10.85%
4. El_Greco – 5.09%
5. Raphael – 3.19%

Predicted: Caravaggio (42.30%)



Processing: claude_monet.jpg



Top predictions:

1. Pierre-Auguste_Renoir – 33.65%
2. Vincent_van_Gogh – 21.58%
3. Alfred_Sisley – 14.81%
4. Claude_Monet – 14.55%
5. Camille_Pissarro – 4.64%

Predicted: Pierre-Auguste_Renoir (33.65%)



Processing: diego_rivera.webp



Top predictions:

1. Pablo_Picasso – 39.63%
2. Vincent_van_Gogh – 19.49%
3. Diego_Rivera – 14.04%
4. Andrei_Rublev – 7.82%
5. Paul_Gauguin – 5.65%

Predicted: Pablo_Picasso (39.63%)



Processing: modigliani_amedeo.jpg



Top predictions:

1. Paul_Gauguin – 19.30%
2. Pierre-Auguste_Renoir – 17.91%
3. Vincent_van_Gogh – 17.32%
4. Pablo_Picasso – 14.31%
5. Sandro_Botticelli – 10.30%

Predicted: Paul_Gauguin (19.30%)



Processing: vincent_van_gogh.jpg



Top predictions:

1. Camille_Pissarro – 33.77%
2. Vincent_van_Gogh – 13.04%
3. Henri_Matisse – 12.36%
4. Paul_Gauguin – 10.13%
5. Henri_Rousseau – 5.15%

Predicted: Camille_Pissarro (33.77%)



Example Predictions on Unseen Artwork

To evaluate the practical usability of the trained model, ten artwork images were selected and classified using the prediction pipeline and Grad-CAM visualization.

The selected images represented a mixture of artists included in the dataset and were not used during model training.

Prediction Summary

Out of the 10 tested images:

- **5 images were classified correctly**
- **5 images were classified incorrectly**

This corresponds to an example prediction accuracy of approximately **60%**, which is consistent with the overall test accuracy achieved during formal model evaluation.

Correct Predictions

The model successfully identified several artists with high confidence, including:

Artist	Confidence
Andy Warhol	99.69%
Camille Pissarro	86.52%
Andrei Rublev	80.33%
Albrecht Dürer	79.33%
Caravaggio	42.30%

These results indicate that the model learned highly distinctive stylistic characteristics for several artists.

Incorrect Predictions

Some images were assigned to visually similar artists:

True Artist	Predicted Artist
Alfred Sisley	Claude Monet
Claude Monet	Pierre-Auguste Renoir
Diego Rivera	Pablo Picasso
Amedeo Modigliani	Paul Gauguin
Vincent van Gogh	Camille Pissarro

Many of these misclassifications are understandable because the involved artists share:

- similar artistic movements,
- comparable color palettes,
- related compositional structures,
- or overlapping stylistic features.

For example:

- Sisley, Monet, and Renoir are all associated with Impressionism.
- Picasso and Rivera both produced highly recognizable figurative compositions.
- Gauguin and Van Gogh share strong Post-Impressionist influences.

Grad-CAM Interpretation

Grad-CAM visualizations revealed that the model focused primarily on meaningful artistic features rather than background regions.

Examples observed during testing include:

- facial structures and portrait details,

- central human subjects,
- brushstroke textures,
- and high-contrast compositional elements.

This behavior suggests that the model learned relevant artistic representations and based its predictions on visually significant regions of the artwork.

Overall Interpretation

The example predictions demonstrate that the model can successfully identify many artists while also exhibiting understandable confusion between stylistically related painters.

Combined with the confusion matrix analysis, these results indicate that the model learned meaningful artistic relationships and generalized reasonably well to previously unseen artwork images.

15. Reflection on project work

Working on this project gave us a deeper understanding of how transfer learning can be applied to fine-grained image classification tasks. We learned how pretrained models like EfficientNetB0 extract meaningful visual features and how techniques such as data augmentation and class weighting help stabilize training on imbalanced datasets. One of the most challenging aspects was dealing with artists whose styles are historically or visually similar, which became clear in the confusion matrix analysis. It was interesting to see that many of the model's mistakes reflected genuine stylistic relationships between artists. Grad-CAM visualizations also helped us understand which image regions influenced the model's decisions. Overall, this project strengthened our understanding of deep learning workflows and highlighted several directions for improvement, such as collecting more balanced data, fine-tuning deeper layers, and experimenting with modern architectures like Vision Transformers.

Note on use of AI

AI was used as a support tool for structuring code and markdown documentation cells. Additionally it was used to help add more comments for better readability.